

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

GIOVANNA REGINA RIBEIRO VITORINO

SHEETMASTERS

Campos do Jordão

2026

GIOVANNA REGINA RIBEIRO VITORINO

SHEETMASTERS

Relatório apresentado ao curso de Tecnologia em Análise e Desenvolvimento de Sistemas, da Faculdade de Campos do Jordão, para projeto proposto à disciplina de Programação Orientada a Objetos.

Orientador: Paulo Giovani de Faria Zeferino.

CAMPOS DO JORDÃO

2026

RESUMO

Este relatório apresenta o processo de desenvolvimento do jogo SheetMasters, um projeto educacional produzido na linguagem C++ utilizando a biblioteca gráfica Raylib. Trata-se de um jogo voltado ao ensino da teoria musical e da leitura de partituras, no qual o jogador interage com exercícios dinâmicos de reconhecimento e escrita de notas musicais. Ao longo da experiência, o usuário navega por dois modos de jogo: incluindo leitura de partitura com percepção auditiva e um quiz teórico. Durante o desenvolvimento, foram implementados sistemas essenciais para a construção da experiência gamificada, incluindo a reprodução sonora automatizada das notas, um sistema de pontuação e gerenciamento de vidas e um algoritmo de dificuldade adaptativa para guiar o progresso do estudante. O projeto busca integrar conceitos de gamificação ao ensino musical, aplicando de forma modular os princípios de programação orientada a objetos (POO) estudados ao longo da disciplina e resultando em uma ferramenta complementar para o aprendizado de música.

Palavras-chave: Jogo, partitura, RayLib.

RESUMO	4
1 INTRODUÇÃO	6
1.1 Objetivos	7
1.2 Justificativa	7
1.3 Aspectos Metodológicos	8
1.4 Aporte Teórico	8
2 METODOLOGIA	9
2.1 Ferramentas Utilizadas	10
2.3 Modelagem UML	10
3 RESULTADOS OBTIDOS	11
3.1 Análise dos Resultados	13
4 CONCLUSÃO	13
4.1 Sugestões de Melhorias	14

1 INTRODUÇÃO

Este trabalho apresenta o desenvolvimento do jogo SheetMasters, um jogo educacional criado em C++ utilizando a biblioteca raylib. O projeto foi desenvolvido com o objetivo de auxiliar estudantes no aprendizado de teoria e percepção musical, transformando o reconhecimento de notas em pauta, a identificação de figuras rítmicas e noções de teoria musical em atividades interativas e progressivas. A narrativa do jogo é centrada na jornada de aprendizado do próprio jogador, que assume o papel de um estudante de música em constante aperfeiçoamento. À medida que avança pelos desafios, novos conteúdos e níveis de dificuldade são desbloqueados, proporcionando uma experiência dinâmica e adaptativa. A mecânica principal baseia-se na resolução de exercícios gerados automaticamente, abrangendo modos de leitura e nomeação de notas, percepção auditiva e questionários teóricos. O sistema utiliza pontuação, sequências de acertos e progressão de dificuldade para manter o desafio adequado ao nível de conhecimento do usuário.

1.1 Objetivos

O objetivo deste trabalho é demonstrar o processo de desenvolvimento de um jogo educacional digital que auxilie no aprendizado de teoria musical por meio de atividades gamificadas. Para alcançar esse propósito, foram aplicados conceitos de Programação Orientada a Objetos no desenvolvimento do jogo, bem como utilizar padrões de projeto e princípios de arquitetura de software que garantam modularidade, organização e facilidade de manutenção do sistema. Além disso, pretende-se desenvolver diferentes modos de jogo voltados à aprendizagem musical, implementar mecanismos de geração automática de exercícios e criar um sistema de dificuldade adaptativa baseado no desempenho do jogador. O projeto também pretende apresentar um feedback visual imediato para reforçar o processo de aprendizagem e integrar recursos gráficos e de áudio utilizando recursos da biblioteca Raylib.

1.2 Justificativa

O desenvolvimento do SheetMasters justifica-se pela necessidade de criar ferramentas complementares que sejam mais atrativas para o ensino musical tradicional, que frequentemente é visto como uma barreira por estudantes iniciantes. A utilização de elementos de gamificação, como pontuação, progressão de níveis, desafios e recompensas, contribui para aumentar o engajamento dos usuários e aumentar a retenção do conteúdo estudado. Além dos aspectos educacionais, o desenvolvimento desse projeto permitiu o aprofundamento em técnicas de programação, como programação orientada a objetos, gerenciamento de estados, manipulação de áudio e implementação de padrões de projeto. Assim, o trabalho une conhecimentos em desenvolvimento de sistemas,

gamificação e educação musical, resultando em uma solução tecnológica com potencial de aplicação no contexto educacional.

1.3 Aspectos Metodológicos

Para o desenvolvimento do jogo, foi utilizada a linguagem C++ (padrão C++17) no editor Visual Studio Code, com compilação via MinGW no Windows, gerenciada pelo sistema de build CMake. A biblioteca Raylib foi responsável pelo controle de tempo, renderização 2D, entrada do usuário e gerenciamento do dispositivo de áudio. O áudio das notas foi gerado de forma completamente procedural por síntese de onda senoidal, eliminando dependências de arquivos externos. O desenvolvimento do projeto foi dividido em etapas: iniciando pela modelagem do domínio musical e das entidades principais (nota, compasso, exercício e clave), foram implementados sucessivamente o renderizador de partituras, o motor de áudio, a fábrica de exercícios e os modos de jogo. A dificuldade adapta-se automaticamente à sequência de acertos do jogador, ampliando o repertório de alturas e introduzindo acidentes conforme a pontuação avança.

1.4 Aporte Teórico

O aporte teórico do projeto se baseia nos princípios da teoria musical básica e em metodologias de aprendizagem baseada na prática (gamificação). No que se refere a tecnologia, o trabalho sustentou-se na documentação oficial da biblioteca Raylib para manipulação de recursos gráficos e sonoros, além das práticas de Programação Orientada a Objetos. O uso de modelagem UML foi essencial para organizar o software de maneira modular, garantindo o desacoplamento de componentes necessário para viabilizar manutenções futuras e a escalabilidade do sistema pedagógico.

Já no campo da teoria musical, foram utilizados conceitos de notação em clave de Sol e Fá, figuras rítmicas, acidentes e armaduras de clave para definir o conteúdo pedagógico dos exercícios. Para realizar a síntese de áudio digital, a documentação da raylib e referências sobre processamento de sinais serviram de base para implementar a geração de amostras PCM em memória, reproduzidas em thread separada para não bloquear o loop principal do jogo.

2 METODOLOGIA

Levantamento de Requisitos - Definição das especificações do sistema, divididas em:

-
- **Funcionais:** Modos de jogo (*ReadAndName* e *QuizMode*), pontuação, sistema de vidas, cronômetro e progressão adaptativa de dificuldade.
- **Não Funcionais:** Execução do áudio em thread separada via `std::thread` para evitar o bloqueio do loop principal de renderização.

Modelagem do Domínio Musical - Estruturação das entidades do núcleo musical através de POO (Programação Orientada a Objetos):

- **Note:** Representa a nota musical (pitch MIDI, valor rítmico, acidente e pausa). Converte o pitch em frequência (\$Hz\$) e calcula a duração em tempos.
- **Measure e Exercise:** Representam, respectivamente, o compasso e o exercício completo.
- **Clef (Classe Abstrata):** Especializada em *TrebleClef* e *BassClef*, definindo a nota de referência na pauta e o *codepoint* Unicode do símbolo gráfico.

Subsistemas de Renderização e Áudio

- **ScoreRenderer:** Centraliza a lógica visual da pauta (linhas, claves, fórmula de compasso, figuras rítmicas, linhas suplementares e acidentes). O posicionamento vertical usa numeração diatônica contínua para mapear o pitch MIDI em um slot relativo à clave.
- **AudioEngine:** Responsável pela síntese sonora em tempo de execução. Localiza o arquivo WAV da nota natural mais próxima nos *assets*, ajusta a frequência via *SetSoundPitch* e reproduz o som de forma assíncrona.

Fábrica de Exercícios e Modos de Jogo

- **ExerciseFactory:** Geração randômica de compassos usando `std::mt19937` com lógica de sacola embaralhada (*shuffled bag*) e trava de anti-repetição (máximo de duas notas iguais seguidas). A dificuldade se expande a cada 10 acertos consecutivos, ampliando o *pool* de alturas e inserindo acidentes.
- **GameMode (Classe Abstrata):** Implementa o *Template Method Pattern*. O método `update()` é final e gerencia o *timer* e os gatilhos (`onUpdate`, `onCorrect`, `onWrong`, `onTimeout`) herdados e sobrescritos pelas classes filhas *ReadAndName* e *QuizMode*.

Integração e Gerenciamento de Estado:

- **GameManager (Singleton):** Centraliza o ciclo de vida da aplicação, controlando a inicialização da janela, carregamento de texturas, injeção de dependências e a máquina de estados (MENU, PLAYING, PAUSED, RESULT). O encerramento

de uma sessão de jogo (fim de vidas ou tempo esgotado) é comunicado ao gerenciador através do callback `onResult`.

2.1 Ferramentas Utilizadas

- Linguagem C++ (padrão C++17);
- Biblioteca Raylib;
- PlantUML;
- Visual Studio Code;
- GitHub;
- CMake;
- MinGW/w64devkit (Windows);

2.3 Modelagem UML

O diagrama de classes UML elaborado com o PlantUML apresenta as relações entre as principais entidades do sistema. As interfaces `IRenderable`, `IUpdatable` e `IAudioSource` definem contratos de renderização, atualização por frame e controle de áudio, respectivamente, e são implementadas pelas classes concretas. A hierarquia de herança parte de `GameMode` em direção aos modos especializados, enquanto a classe `Clef` origina `TrebleClef` e `BassClef`. As relações de composição são visíveis entre `GameManager` e suas dependências (`ScoreRenderer`, `AudioEngine` e `ExerciseFactory`), bem como entre `Exercise`, `Measure` e `Note`. O diagrama evidencia o alto grau de modularidade do sistema e o baixo acoplamento entre seus componentes.

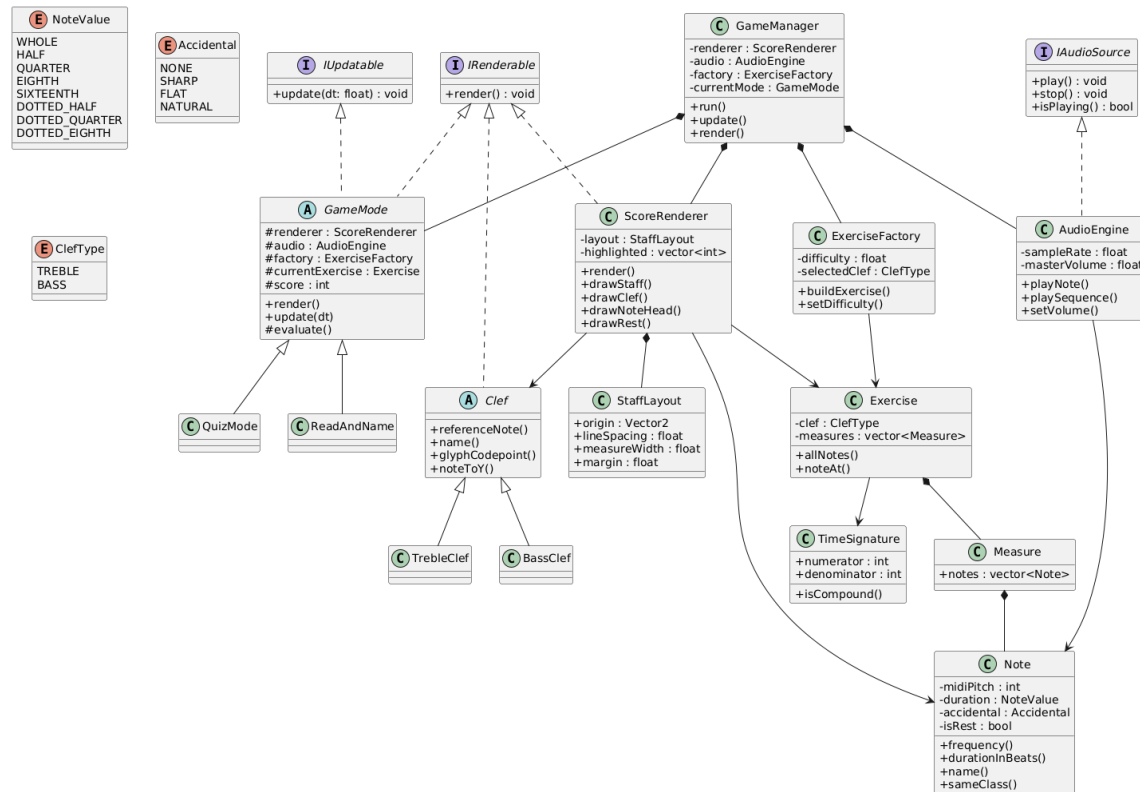


Figura 1 – Diagrama UML do projeto SheetMasters.

GameManager: coordena execução do jogo. GameMode: abstração para modos de jogo. QuizMode e ReadAndName: modos especializados. ExerciseFactory: gera exercícios. ScoreRenderer: renderiza partituras. AudioEngine: reproduz notas. Note, Measure e Exercise representam o domínio musical.

O projeto utiliza encapsulamento para proteger atributos internos, herança para especialização dos modos de jogo, polimorfismo por meio de interfaces e abstração para representar conceitos musicais complexos.

3 RESULTADOS OBTIDOS

Ao término do desenvolvimento, o SheetMasters apresentou resultados funcionais em os módulos planejados. A seguir, são descritas as principais telas e funcionalidades implementadas, acompanhadas das respectivas capturas de tela para ilustrar a experiência do usuário.



Figura 2 – Tela do menu principal

A tela inicial apresenta ao jogador os modos disponíveis, a pontuação total acumulada entre sessões e o recorde de sequência de acertos, permitindo também selecionar a clave de Sol ou a clave de Fá antes de iniciar.

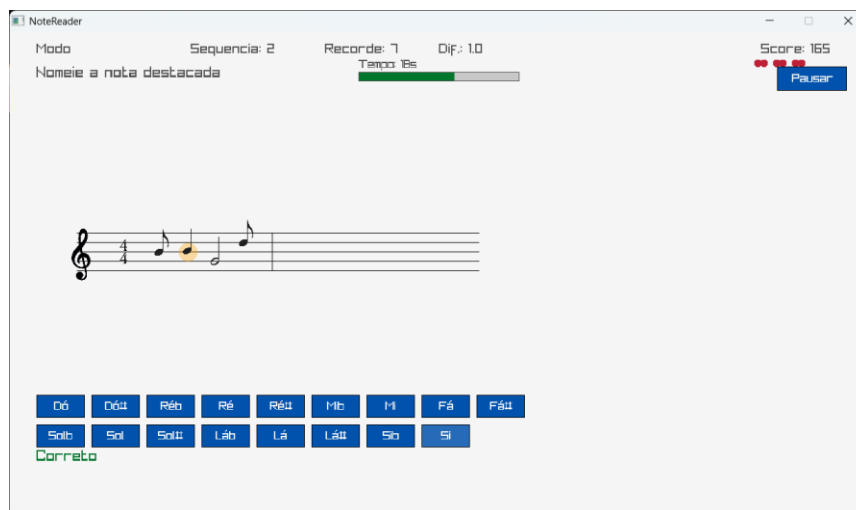


Figura 3 – Tela do modo Ler e Nomear Notas (ReadAndName).

A pauta é renderizada com a nota destacada em laranja e os botões de resposta dispostos na parte inferior. O HUD exibe a sequência de acertos, o nível de dificuldade, o cronômetro e as vidas restantes. O usuário deve identificar a nota destacada e clicar no

nome correspondente. A cada acerto, o sistema avança para a próxima nota e toca o som correspondente via AudioEngine; a cada erro, uma vida é descontada.

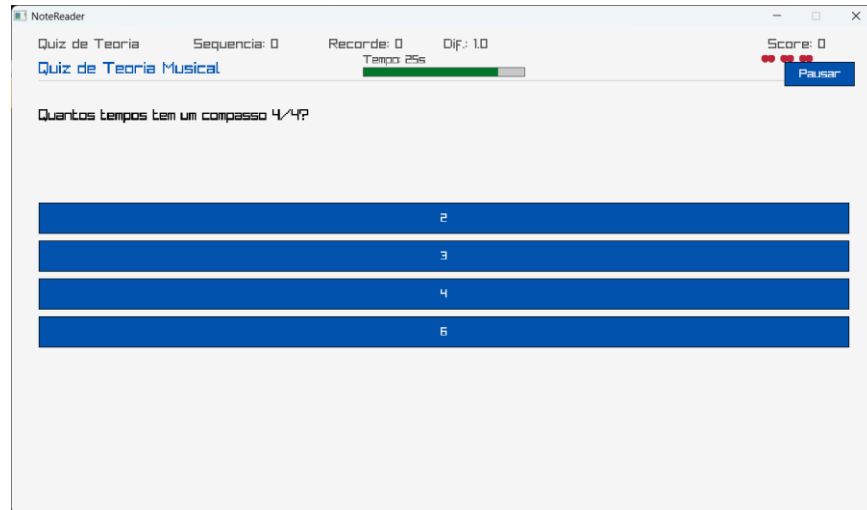


Figura 4 – Tela do modo Quiz de Teoria Musical (QuizMode).

A pergunta é exibida no topo e as quatro alternativas aparecem como botões na área central. O banco contém 17 questões sobre teoria musical, sorteadas aleatoriamente a cada rodada.

3.1 Análise dos Resultados

Os testes funcionais realizados ao longo do desenvolvimento confirmaram a correta operação de todos os subsistemas. O gerador de exercícios produziu compassos válidos respeitando a fórmula de compasso 4/4, sem extrapolar o total de tempos por compasso. O renderizador de partitura posicionou as notas com precisão diatônica tanto na clave de Sol quanto na clave de Fá, incluindo linhas suplementares para notas fora da pauta. O sistema de áudio reproduziu corretamente as notas sem bloquear o loop gráfico, graças ao uso de `std::thread` com sincronização por `std::atomic`. O mecanismo de progressão adaptativa elevou a dificuldade de forma gradual, ampliando o repertório de alturas e introduzindo acidentes após dez acertos consecutivos. O sistema de vidas e a expiração de tempo funcionaram conforme o esperado, acionando corretamente o callback `onResult` e a transição para o estado `RESULT` ao final de cada sessão.

4 CONCLUSÃO

O desenvolvimento do SheetMasters permitiu a aplicação prática e integrada dos principais conceitos de Programação Orientada a Objetos em um contexto educacional com valor pedagógico real. Ao longo do projeto, foram implementados encapsulamento,

herança, polimorfismo e abstração de forma coerente com os requisitos do domínio musical, resultando em uma arquitetura modular e de fácil manutenção.

O padrão Template Method aplicado em GameMode demonstrou ser adequado para a abstração do fluxo comum de atualização e pontuação, permitindo que novos modos de jogo sejam adicionados com mínima alteração no código existente. O uso do padrão Singleton para o GameManager garantiu que recursos críticos, como a janela gráfica e o dispositivo de áudio, fossem inicializados e destruídos de forma controlada. A separação entre a lógica de geração de exercícios (ExerciseFactory), renderização (ScoreRenderer) e reprodução de áudio (AudioEngine) conferiu alta coesidade a cada módulo e facilitou os testes individuais de cada subsistema.

Do ponto de vista educacional, o jogo alcançou o objetivo de transformar o reconhecimento de notas musicais em atividade interativa, com feedback imediato, progressão adaptativa e reforço auditivo, tornando o aprendizado mais dinâmico e engajante em comparação com métodos tradicionais. Os resultados obtidos durante os testes funcionais confirmaram a viabilidade da proposta como ferramenta complementar para o ensino musical.

4.1 Sugestões de Melhorias

Com base nas limitações identificadas durante o desenvolvimento e os testes, propõem-se as seguintes melhorias para versões futuras do projeto: (a) implementação de um modo de ditado musical, no qual o usuário ouve uma sequência de notas e deve transcrevê-la na pauta, ampliando o treinamento auditivo; (b) persistência de dados entre sessões, com armazenamento local do histórico de desempenho e recorde, possibilitando acompanhar a evolução do estudante ao longo do tempo; (c) suporte a armaduras de clave, integrando escalas maiores e menores aos exercícios e ampliando o conteúdo pedagógico abordado; (d) integração com dispositivos MIDI externos, permitindo que o usuário responda aos exercícios tocando diretamente em um instrumento conectado ao computador; (e) suporte a novas claves, como clave de Dó;

REFERÊNCIAS

RAYLIB. Raylib — a simple and easy-to-use library to enjoy videogames programming. 2025. Disponível em: <https://www.raylib.com/>. Acesso em: 15 abr. 2026.